

TP : Introduction à Kubernetes – Fonctionnement et Manipulations

1. Introduction

Ce TP a pour objectif de découvrir **Kubernetes**, une plateforme d'orchestration de conteneurs largement utilisée dans les architectures cloud modernes.

À travers des rappels théoriques et des manipulations pratiques, ce travail vise à comprendre comment Kubernetes permet de **déployer, gérer et mettre à l'échelle des applications conteneurisées** de manière automatisée.

Les manipulations réalisées illustrent les concepts fondamentaux tels que les **Pods**, **deployments**, **services** et la communication entre composants.

2. Rappels de cours : conteneurisation et orchestration

2.1 Conteneurisation

Un **conteneur** est une unité logicielle légère qui regroupe :

- une application
- ses dépendances
- sa configuration

Contrairement aux machines virtuelles, les conteneurs partagent le noyau du système hôte, ce qui les rend :

- plus rapides à démarrer
- moins gourmands en ressources

Docker est l'outil de conteneurisation le plus utilisé avec Kubernetes.

2.2 Orchestration de conteneurs

L'orchestration consiste à gérer automatiquement :

- le déploiement des conteneurs
- leur redémarrage en cas de panne
- la montée en charge (scaling)
- la communication réseau

Kubernetes est aujourd'hui le standard de facto pour l'orchestration.

3. Architecture de Kubernetes

3.1 Cluster Kubernetes

Un **cluster Kubernetes** est composé de plusieurs nœuds :

- **Master (Control Plane)**
Responsable de la gestion globale du cluster :
 - API Server
 - Scheduler
 - Controller Manager
 - etcd (base de données)
 - **Worker Nodes**
Exécutent les applications sous forme de pods.
-

3.2 Objets fondamentaux de Kubernetes

- **Pod** : plus petite unité de déploiement
 - **Deployment** : gère les replicas et les mises à jour
 - **Service** : expose une application
 - **Namespace** : isolation logique
 - **ConfigMap / Secret** : gestion de configuration
-

4. Fonctionnement général de Kubernetes

1. L'utilisateur décrit l'état souhaité (fichier YAML)
2. Kubernetes compare l'état réel à l'état désiré
3. Le contrôleur agit pour corriger les écarts
4. Le cluster converge vers l'état demandé

Principe clé : Kubernetes fonctionne selon un **modèle déclaratif**.

5. Manipulations réalisées

5.1 Démarrage d'un cluster Kubernetes

Dans un environnement local, un cluster peut être lancé avec :

- Minikube
- Kind
- Docker Desktop

Commande typique :

```
minikube start
```

Interprétation :

Cette commande crée un cluster Kubernetes local, permettant de tester des déploiements sans infrastructure réelle.

5.2 Création d'un Pod

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: mon-pod

spec:

  containers:

    - name: nginx

      image: nginx
```

Commande :

```
kubectl apply -f pod.yaml
```

Interprétation :

Le pod contient un conteneur Nginx. Kubernetes s'assure qu'il est lancé et reste actif.

5.3 Déploiement avec un Deployment

```
apiVersion: apps/v1

kind: Deployment

metadata:

  name: nginx-deployment

spec:

  replicas: 3

  selector:

    matchLabels:

      app: nginx

  template:

    metadata:

      labels:
```

```
    app: nginx

spec:

  containers:

    - name: nginx

      image: nginx
```

Commande :

```
kubectl apply -f deployment.yaml
```

Interprétation :

- Trois pods identiques sont créés
- En cas de panne d'un pod, Kubernetes le recrée automatiquement

5.4 Vérification de l'état

Commandes utilisées :

```
kubectl get pods
```

```
kubectl get deployments
```

```
kubectl describe pod mon-pod
```

Interprétation :

Ces commandes permettent de surveiller l'état du cluster et de diagnostiquer d'éventuels problèmes.

5.5 Exposition d'une application avec un Service

```
apiVersion: v1
```

```
kind: Service

metadata:
  name: nginx-service

spec:
  type: NodePort

  selector:
    app: nginx

  ports:
    - port: 80
      targetPort: 80
```

Commande :

```
kubectl apply -f service.yaml
```

Interprétation :

Le service permet d'accéder à l'application depuis l'extérieur du cluster.

5.6 Mise à l'échelle (Scaling)

```
kubectl scale deployment nginx-deployment --replicas=5
```

Interprétation :

Kubernetes crée automatiquement de nouveaux pods pour répondre à la demande.

6. Avantages de Kubernetes

- Haute disponibilité

- Auto-réparation
- Scalabilité
- Portabilité cloud
- Déploiements automatisés

Interprétation :

Kubernetes facilite la gestion d'applications complexes dans des environnements distribués.

7. Limites et contraintes

- Courbe d'apprentissage élevée
- Complexité de configuration
- Besoin de surveillance et de sécurité